



Deploying Low-Latency APIs for Real-Time AI

Sub-100ms decisions for ads, fraud, personalization

Design trade-offs: speed ⚡ vs accuracy 🎯 vs cost 💰

APIs are the front door of real-time AI

Why API Latency Matters

Lost Revenue

Slow ad auction API → lost bid,
lost revenue

Fraud Risk

Fraud scoring API >50ms →
transaction already processed

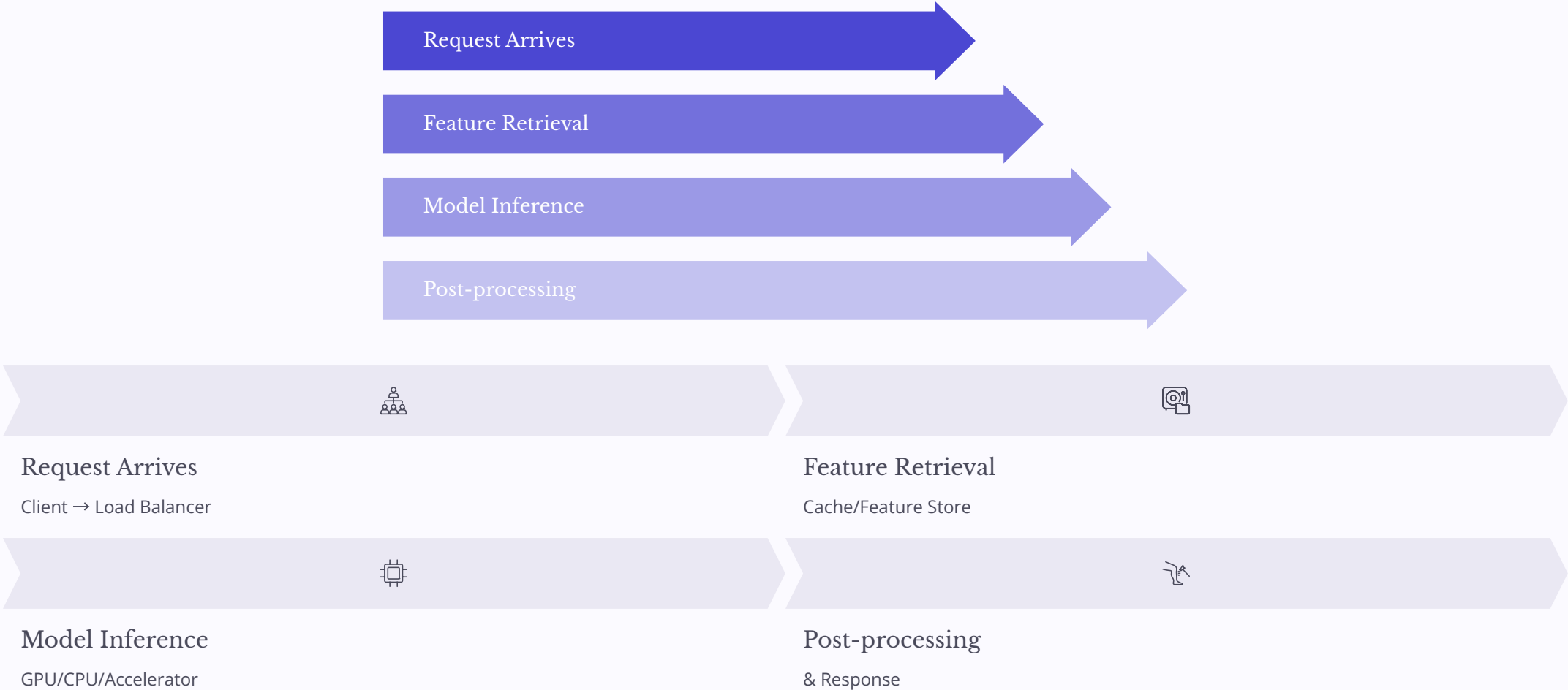
User Experience

Personalized feed >200ms → user churn



Latency is both a **UX issue** and a
business risk

Anatomy of a Real-Time API Call



Target SLA: 50-100ms total

Key Infrastructure Requirements

1 Autoscaling

Scale pods/instances based on QPS (queries per second)

Prevent degradation during traffic spikes

2 Warm Pools

Avoid cold starts (especially in serverless)

Maintain pre-initialized compute resources

3 Multi-Region Deployment

Serve users from closest geographic location

Reduce network latency by 30-100ms

4 Monitoring

Track latency histograms, P95/P99 alerts

Identify performance regressions early



Every millisecond matters when scaling to millions of API requests per second

Model Serving Frameworks



Triton Inference Server

NVIDIA's GPU-optimized server

Multi-framework support (TF, PyTorch, ONNX)

Dynamic batching capabilities



TensorRT / ONNX Runtime

Accelerate deep neural networks

Kernel fusion and optimization

Up to 5x speedup for inference



KServe

Kubernetes-native serving

Scale to thousands of models

Declarative deployment APIs



BentoML

Startup-friendly packaging

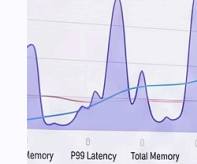
Model → API streamlined workflow

Python-first developer experience

TensorFlow Serving

Requests per Second (RPS)

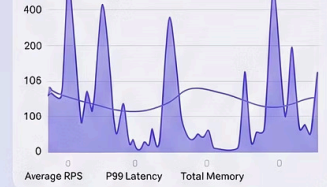
Latency (ms)



TorchServe

Requests per Second (RPS)

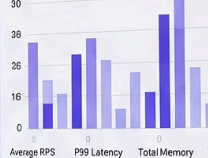
Latency (ms)



Triton Inference Server

Requests per Second (RPS)

Latency (ms)



Latency Optimization Techniques

Model Optimization

Quantization & distillation → smaller, faster models

FP16/INT8 precision reduces compute requirements

Knowledge distillation compresses model size

Data Optimization

Feature caching → reduce DB/feature store lookups

In-memory caches for frequently accessed features

Prefetching for predictable access patterns

Compute Optimization

Batching/microbatching → amortize inference cost

Group requests to maximize GPU/CPU utilization

Dynamic batch sizing based on queue depth

Network Optimization

Edge inference → push models closer to users

CDN-like distribution of smaller models

Reduce round-trip time by 50-150ms



- ❏ A 25ms reduction in latency can translate to millions in additional revenue for high-frequency trading or ad platforms

Example: FastAPI for Model Serving

```
from fastapi import FastAPI
import torch
from model import Net

app = FastAPI()
model = torch.load("model.pt")
model.eval()

@app.post("/predict")
def predict(x: list[float]):
    with torch.no_grad():
        return {
            "prediction": model(
                torch.tensor(x)
            ).argmax().item()
        }
```

Advantages

- Lightweight, async-capable
- Simple developer experience
- Fast deployment cycles
- Native OpenAPI documentation

Limitations

- Not GPU-optimized by default
- Limited batching capabilities
- Requires custom scaling solution
- Model reloading on each instance

Best for: Prototype APIs and smaller production workloads

Example: Triton Deployment (YAML)

```
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: fraud-detector
spec:
  predictor:
    triton:
      runtimeVersion: 23.05-py3
      storageUri: s3://models/fraud-detector/
      resources:
        requests:
          nvidia.com/gpu: 1
```

KServe + Triton = scalable low-latency serving

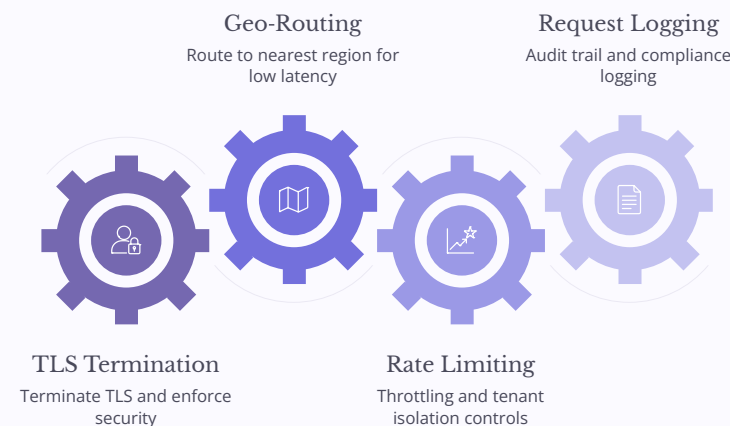
Automatic resource scaling based on inference demand



The deployment creates a fully managed inference endpoint with:

- Automatic GPU allocation
- S3/cloud storage model artifacts
- Load balancing across instances
- Health checking and auto-recovery

API Gateway & Load Balancer Role



Security

- Terminate TLS connections
- Handle authentication and authorization



Routing

- Route to nearest region (geo-routing)
- Reduce network latency by 30-100ms



Traffic Management

- Enforce rate limiting and quotas
- Maintain tenant isolation



Compliance

- Log every request for audit trails
- Support regulatory requirements

Monitoring Latency in APIs

Key Metrics to Track

→ Latency Percentiles

P50, P95, P99 latencies

Captures real user experience better than averages

→ Throughput

Requests per second (RPS/QPS)

Track capacity and utilization trends

→ Error Rates

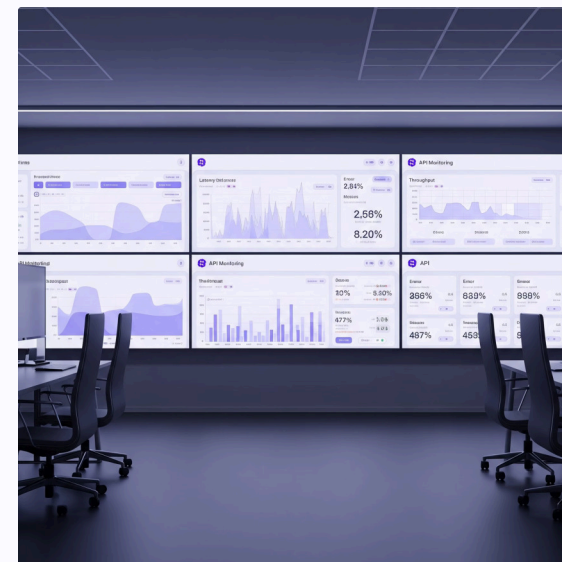
4xx client errors, 5xx server errors

Track error budget consumption

→ Resource Utilization

CPU, memory, GPU utilization

Identify bottlenecks and optimization opportunities



Recommended Tooling

- Prometheus + Grafana for metrics collection and visualization
- Datadog APM for distributed tracing and service maps
- Cloud-native observability dashboards (CloudWatch, StackDriver)
- [OpenTelemetry](#) for vendor-neutral instrumentation

Case Study: Ad-Tech API



✓ Business Impact

+12% auction win rate

+8.5% revenue increase

ROI on optimization: 315%

Challenge: 220ms Latency = Lost Auctions

Initial State

1

- Ad bidding API: 220ms average latency
- Missing auction deadlines (150ms cutoff)
- Significant revenue opportunity loss
- Complex ML models with large feature sets

2

Optimization Strategy

- Implemented in-memory feature cache
- TensorRT optimized bidding model
- Pre-computed common feature transformations
- Deployed to edge locations near exchanges

Results

3

- 75ms average latency (66% reduction)
- 99.5% of requests under 150ms deadline
- +12% auction win rate
- Technical debt reduced by 40%

Case Study: Banking Fraud API

Challenge: Slow Fraud Detection = Missed Fraud



Initial State

CPU-only deployment

180ms average latency

Missing SLA targets

High false negative rate



Results

45ms average latency (75% reduction)

100% SLA compliance

15% fraud detection improvement

"The latency improvement allowed us to implement more sophisticated models without sacrificing response time, significantly improving our fraud detection capability."



Solution

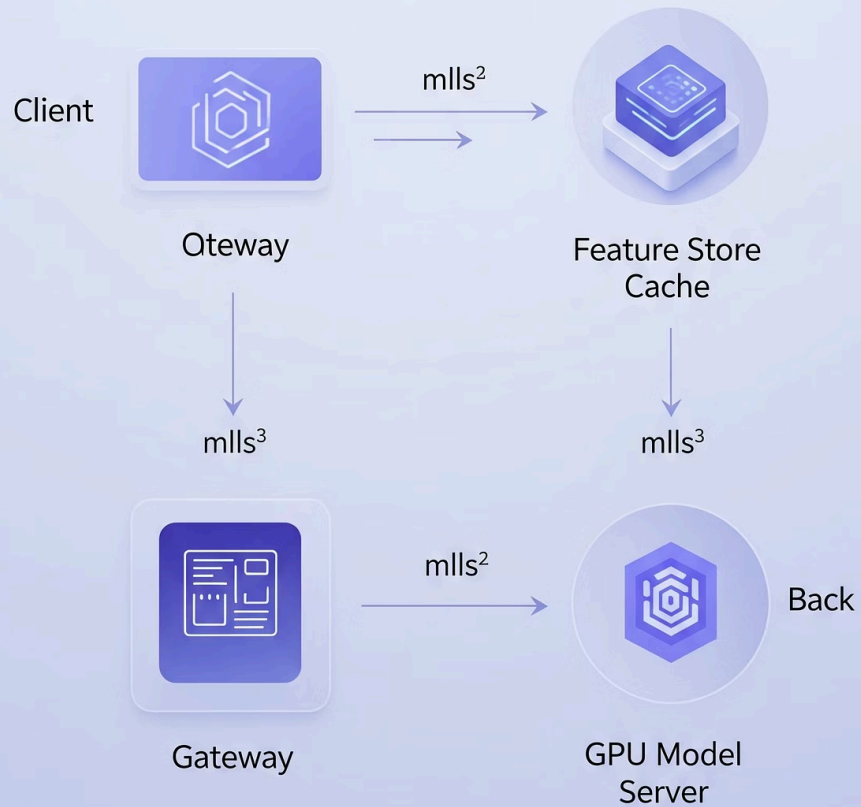
Migrated to Triton + GPUs

Implemented microbatching

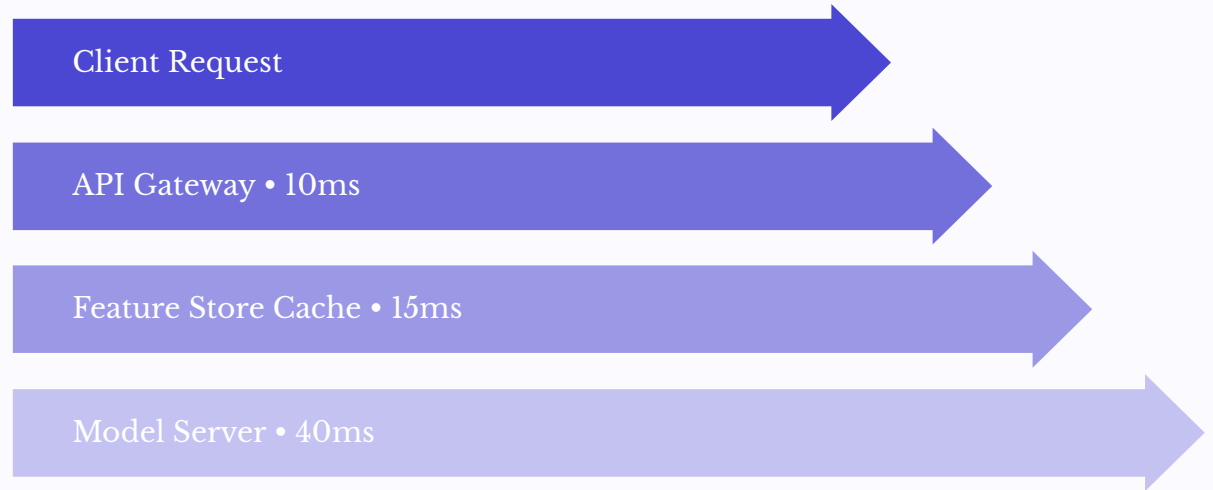
Model ensemble with prioritization



API Call flow



API Design is Latency Design



"In real-time AI, milliseconds create competitive advantage."

Summary: Building Sub-100ms APIs

Key Takeaways

- Low-latency APIs = lifeline of real-time AI applications
- Infrastructure needs: autoscaling, caching, GPU inference servers
- Optimizations: quantization, batching, edge deployment
- Monitor P95/P99 latencies, not just averages

Next Steps

1. Assess current API latency (baseline metrics)
2. Identify critical paths and bottlenecks
3. Implement quick wins (caching, connection pooling)
4. Evaluate specialized inference servers



Remember: Every component in your API stack must be designed with latency in mind, from infrastructure to model architecture

Questions?